TPLP: Page 1–8. © The Author(s), 2021. Published by Cambridge University Press 2021
doi:10.1017/xxxxx

A Datalog Framework for Conflict-Free Replicated Data Types (Extended Version)

Elena Yanakieva
RPTU University Kaiserslautern-Landau

Annette Bieniusa
RPTU University Kaiserslautern-Landau

Stefania Dumbrava
ENSIIE, Inria Paris, SAMOVAR (Télécom SudParis)

Abstract

Distributed applications increasingly support local-first collaboration over shared data, allowing multiple users to perform updates concurrently without global coordination. Such collaboration requires careful design in order to capture the intended semantics of the concurrent interactions.

We introduce a declarative framework for specifying and reasoning about the semantics of conflict-free replicated data types (CRDTs) and CRDT-based applications in Datalog. The framework models CRDT semantics as executable logic programs over operation contexts, making concurrency explicit, compositional, and amenable to automated analysis. As one application, we use property-based testing to compare implementations. To the best of our knowledge, this is the first work to systematically use Datalog as a foundation for prototyping and analyzing complex CRDTs and their compositions.

We evaluate our methodology using a collaborative graph data editing case study and report experimentation results assessing correctness validation and scalability with an increasing number of operations and replicas.

KEYWORDS: logic programming, conflict-free replicated data types, local-first, CRDTs

1 Introduction

Distributed applications increasingly support collaboration over shared data, allowing multiple users to update the same data concurrently. This raises a fundamental question: when concurrent updates conflict, to which state should the application converge? To ensure predictable and well-defined behavior, developers must therefore specify application-specific conflict semantics. From a semantic perspective, the core challenge is, thus, not only convergence itself but also the lack of explicit, analyzable specifications that explain why a particular concurrent outcome arises.

This challenge is amplified by the need for high availability. To enable offline work and low-latency local updates, collaborative systems must weaken consistency guarantees (Gilbert and Lynch 2002) and reconcile concurrent updates afterwards. Conflict-free

Replicated Data Types (CRDTs) address this need by providing replicated data structures with deterministic convergence and strong eventual consistency (SEC) (Shapiro et al. 2011). However, building applications from CRDTs remains difficult in practice. Developers typically combine multiple data structures, but conflict-resolution policies do not compose predictably. Subtle concurrency patterns can cause a correct CRDT implementation to converge to an application-level state that violates intended semantics. While CRDTs guarantee deterministic convergence, they do not guarantee that the converged state reflects the application's intended semantics when CRDTs are composed.

Our work addresses this problem by providing a declarative modeling framework and tool that support developers in exploring the composability of CRDT semantics.

Related Work. Dedalus (Alvaro et al. (2010)) introduces a temporal Datalog-based formalism for reasoning about replicated mutable state and asynchronous communication in long-running distributed systems, whereas our focus is on executable semantic specifications for replicated data types and their compositions. It incorporates time as a first-class attribute of predicates and supports stratified reasoning with negation over time, providing a model-theoretic foundation for distributed program semantics. Building on these ideas, the Bloom language (Alvaro et al. (2011)) proposes a declarative framework for reasoning about distributed programs through monotonic programming over sets, assuming a fixed merge operator (union) and a containment-based partial order.

Subsequent systems explored deterministic and coordination-free programming models using lattice-based structures. LVars (Kuper and Newton (2013)) introduced deterministic-by-construction parallelism over lattices, while Lasp (Meiklejohn and Roy (2015)) proposed a functional programming model for composing state-based CRDTs into larger computations that satisfy SEC.

More recently, the CALM theorem (Laddad et al. (2022)) provided a unifying theoretical result: programs whose outputs are logically monotonic can achieve consistency without coordination. This insight has since inspired automated analyses aimed at identifying when distributed programs can avoid coordination. Because logical monotonicity in Datalog can often be conservatively inferred from syntactic structure, Datalog remains a natural foundation for such reasoning.

Soundarapandian et al. (2022) proposes a framework for certified mergeable replicated data types, in which CRDT implementations are accompanied by machine-checked proofs of convergence and correctness. Their approach builds MRDTs in the F* verification framework and discharges correctness proofs via SMT solving. While this work focuses on certifying the correctness of individual CRDT designs and merge functions, it does not target application-level semantic correctness across compositions of CRDTs.

In summary, existing formal models of replicated data types provide limited support for assessing whether composed implementations realize an application's intended semantics. Certified CRDT frameworks focus on proving convergence or functional correctness of a given design, typically assuming its semantic correctness. In contrast, we aim to make the semantic mismatches between the intended behavior and the CRDT compositions explicit, executable, and falsifiable.

Our approach. To avoid unintended behavior, developers need to be able to state the intended semantics of an application independently of a particular CRDT design. We

propose a workflow in which designers first write a declarative, executable reference specification of the application semantics and then construct a realization via CRDT composition. Our framework supports systematic testing: given generated concurrent executions, it checks whether a composite realization converges to the same state as the reference specification using property-based testing. While formal verification of distributed applications remains expert-level and costly, executable specifications enable lightweight, systematic testing readily usable during design.

Datalog is central to our work. First, it provides a concise and declarative way to express CRDT semantics as relations over events and visibility. Second, its monotonic, bottom-up evaluation model aligns naturally with CRDT semantics, where state is derived deterministically from a growing set of observed events. Third, Datalog specifications are executable, enabling systematic testing without committing to a concrete distributed implementation.

We implement the framework in CRDTLog, a Datalog-based system built on the Soufflé engine. CRDTLog provides a library of declarative specifications for basic, nested, and composite CRDTs, as well as tools to generate operation contexts and compare semantic outcomes. *To the best of our knowledge, this is the first work to systematically use Datalog as a foundation for prototyping and analyzing complex CRDTs and their compositions.* We make the following contributions:

- We present an extended declarative framework for specifying the semantics of CRDTs and collaborative applications based on operation contexts.
- We implement the framework in Soufflé and provide a library of classical, nested, and composite CRDT specifications, including two CRDT-based composite realizations of collaborative graph applications.
- We present a case study on collaborative graphs that demonstrates how abstract semantic specifications guide CRDT composition choices and expose non-obvious concurrency behaviors.
- We evaluate the approach empirically, assessing semantic equivalence between abstract specifications and composite realizations across a set of generated executions.

2 Background: Collaborative Data Structures

Our work is motivated by *collaborative applications* that require data structures to be shared and updated by multiple users concurrently under weak connectivity and without global coordination. As an example, consider a (directed) graph (V, A) where V is a set of nodes and $A \subseteq V \times V$ is a set of directed edges, as used in a collaborative design canvas or shared knowledge bases. Users may modify the graph by adding (and removing) nodes and edges; to simplify the presentation, we focus here on the structural updates and do not consider modifications of labels. Table 1 summarizes the semantics of such a graph with *isolate-delete (ID)* semantics.

Additionally, we require an application-level invariant, which guarantees the absence of dangling edges: $\forall (u, v) \in A : \; u \in V \wedge v \in V$.

In a single-user, sequential setting, this invariant can be enforced directly by the operations. However, under concurrent updates by multiple users, preconditions are not stable leading to invariant violations. We illustrate such a violation in Figure 1. Consider

Operation	**Precondition**	**Postcondition**	**Return value**
`hasN`(v)	-	$V' = V,\ A' = A$	$v \in V$
`hasE`(u,v)	-	$V' = V,\ A' = A$	$(u, v) \in A$
`addN`(v)	-	$V' = V \cup \{v\},\ A' = A$	void
`addE`(u,v)	$u \in V \wedge v \in V$	$A' = A \cup \{(u, v)\},\ V' = V$	void
`rmvE`(u,v)	-	$A' = A \setminus \{(u, v)\}, V' = V$	void
`rmvN`(v)	$v \in V \wedge \forall x \in V : (x, v) \notin A \wedge (v, x) \notin A$	$V' = V \setminus \{v\},\ A' = A$	void

Table 1: Operations for a directed graph with isolate-delete (ID) semantics; only isolated, non-connected nodes may be removed.

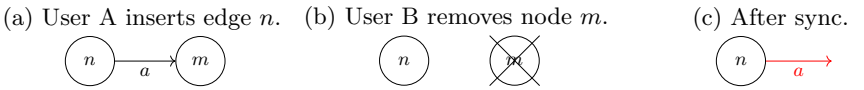


Fig. 1: Scenario: Dangling edge.

a graph with two nodes n and m. User A executes `addE`(n,m) while user B concurrently executes `rmvN`(n). To avoid an invalid graph state, a conflict-resolution strategy is required to preserve the invariant. One possible outcome is to remove n and discard edge a; another is to discard the removal of n. No resolution is universally correct – the appropriate behavior depends on the intended application semantics.

Developers must therefore specify how conflicting operations should be interpreted and combined while preserving availability to enable offline execution and immediate local reasoning. The CAP theorem states that, in the presence of network partitions, strong consistency and availability cannot be guaranteed simultaneously (Gilbert and Lynch (2002)). Consequently, local-first collaborative systems require an explicit and declarative choice of *conflict semantics*.

Conflict-free Replicated Data Types (CRDTs) provide a standard approach to implementing highly available replicated state with deterministic convergence (Shapiro et al. 2011; Borth et al. 2025). Informally, CRDTs guarantee strong eventual consistency (SEC): once all replicas have received the same set of updates, they converge to the same state. Several systems expose CRDTs to application developers, including Yjs (Nicolaescu et al. 2015) and Automerge (Klepmann 2017). These frameworks provide a collection of basic replicated data types, such as sets, maps, sequences, counters, and registers, each equipped with a specific conflict-resolution policy.

Throughout this paper, we refer to common policies for datatypes such as:

- *add-wins* sets: concurrent `add`(v) and `del`(v) leave `v` present;
- *update-wins* maps: concurrent `upd`(k,v)and `rmv`(k) preserve the binding for `k` when the entry is concurrently updated.

These policies are useful primitives, but they are not neutral; each one encodes a specific choice about user intent under concurrency.

Real applications rarely consist of a single basic CRDT. Instead, developers compose and nest CRDTs to obtain richer behavior. However, conflict resolution does not auto-

matically compose intuitively (Kuessner et al. 2023): combining CRDTs can introduce unintended behaviors that may only surface under subtle concurrent execution patterns.

This motivates a tool-supported workflow in which developers specify application semantics abstractly, explore alternative CRDT decompositions, and verify that the chosen design matches the intended behavior. Ideally, this relies on an executable specification enabling systematic exploration of concurrent user interactions. The remainder of the paper develops such a framework and implements it in Soufflé (Jordan et al. 2016b).

Our framework builds on the declarative model of replicated data types by Burckhardt (2014), which defines each data type as a deterministic function over an operation context, in which events and their relation are captured.

Definition 1 (*Operation Context and Replicated Data Type*)
Let E be a set of events, Op_X be the set of operations for a replicated data type X and $Values$ be a set of possible return values. An *operation context* is a finite event graph $C = (\mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar})$, where $\mathrm{op} : \mathrm{E} \to Op_X$ describes the operation of each event, vis is an acyclic relation representing visibility among the elements of E, and ar is a total order representing the arbitration of the elements in E.

A *replicated data type* $\mathcal{F}_{\mathrm{X}} : Op_X \times C \to Values$ is a (deterministic) function that, given an operation o and an operation context C, specifies the expected return value $\mathcal{F}_{\mathrm{X}}(o, C)$ when performing operation o in context C.

$\mathcal{F}_{\mathrm{X}}$ is a declarative specification that maps an operation context to the result of each operation. Unlike Burckhardt (2014), our model restricts contexts to update operations. Queries (e.g., $\texttt{hasN}(\mathrm{v})$) are treated as post-hoc observers evaluated over the context to obtain the final state. Although concurrency is modeled via the visibility relation, the evaluation of $\mathcal{F}_{\mathrm{X}}$ is deterministic.

In an add-wins set, an element v is present if there exists an $\texttt{add}(v)$ event e_1 and no $\texttt{del}(v)$ event e_2 that observes it (i.e., e_1 is not visible to e_2). Thus, a delete concurrent with the add does not remove v, capturing add-wins semantics. Formally:

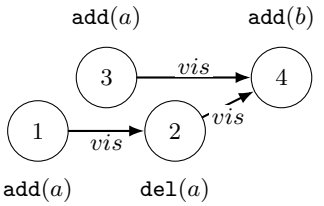
$$\begin{aligned} &\mathcal{F}_{\mathrm{SetAW}}\big(\texttt{hasV}(v), (\mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar})\big) = \mathrm{true} \\ &\quad \iff \exists e \in E : \mathrm{op}(e) = \texttt{add}(v) \ \wedge \neg\big(\exists e' \in E : \mathrm{op}(e') = \texttt{del}(v) \wedge e \xrightarrow{\mathrm{vis}} e'\big) \end{aligned} \tag{1}$$

Figure 2 illustrates an example operation context for an add-wins set. The context contains four events: event 1 performs $\texttt{add}(a)$, event 2 $\texttt{del}(a)$, event 3 $\texttt{add}(a)$, and event 4 performs $\texttt{add}(b)$. The visibility relation includes edges $1 \to 2, 3 \to 4$ and $2 \to 4$, meaning that event 2 observes event 1; event 4 observes events 3, 2, and transitively also 1. Events 3 and 4 are concurrent with events 1 and 2. As arbitration order is irrelevant for an add-wins set, it is omitted. The resulting set contains a and b, since the concurrent $\texttt{add}(a)$ in event 3 overrules $\texttt{del}(a)$ in event 2.

3 A Declarative Framework for CRDT Specification

We introduce CRDTLog[1], an implementation of the declarative approach to CRDTs from Sec. 2. It extends the framework with nested and composite CRDTs, enabling designers to model CRDT-based applications and reason about their semantics."

[1] CRDTLog's code is available at `https://github.com/elly-yanakieva/SouffleCRDTs`.




```
setEvent(1,add,a). setEvent(2,remove,a).
setEvent(3,add,a). setEvent(4,add,b).
vis(1,2). vis(3,4). vis(2,4).

visible(E1,E2) :- vis(E1,E2).
visible(E1,E2) :- visible(E1,E3), vis(E3,E2).
conc(E1,E2) :- setEvent(E1,_,_), setEvent(E2,_,_),
        not visible(E1,E2), not visible(E2,E1),
    E1/=E2.
```

Fig. 2: Example operation context for an add-wins set and its Datalog encoding.

Datatype	**Update operations**	**Query operations**
Set	add(v), del(v)	hasV(v)
MapOfSet	upd(k,add(v)), upd(k, del(v),,) rmv(k)	hasKV(k,v)
Graph	addN(n), rmvN(n), addE(n,m), rmvE(n,m)	hasN(n), hasE(n,m)

Table 2: Summary: Datatype operations.

CRDTLog is implemented in Soufflé, a Datalog engine with efficient, scalable bottom-up evaluation Jordan et al. (2016a). For readability, we present the theory and its Datalog implementation side by side: we represent operation contexts as input relations and derive return values (for queries) as output relations.

Our methodology (Fig. 3) begins with a specification of the *specification level semantics (SLS)* of an application, which defines the intended behavior under concurrency. The developer then decomposes the application into CRDT components and defines transformation rules that map each application event to the operations on these components. CRDTs such as sets and registers serve as basic building blocks; more complex data types can be formed by nesting and composing these blocks. Transformation rules then specify how application events affect each component and, crucially, which component-level conflicts are induced. Together, the components and transformation rules yield the application's *implementation compositional semantics (ICS)*. Since both SLS and ICS are executable, the developer can validate their semantic equivalence by testing them on the same operation contexts and check desired properties at either level. If equivalence fails, the decomposition and rules are revised and the process iterates.

Throughout this section, we refer to the collaborative graph introduced in Sec. 2 and summarized in Table 1 as a running example to illustrate the workflow. The operations of the used datatypes can be found in Table 2.

3.1 Operation Contexts in Datalog

We encode an operation context in CRDTLog using three input relations: one for events and their operations, `vis` for the visibility relation, and `ar` for the arbitration order.

CRDT systems typically derive visibility, a.k.a. happens-before, from metadata such as vector clocks, and arbitration order from timestamps. The arbitration order is used only when semantics require a total order on concurrent events (e.g., last-writer-wins); otherwise, conflict resolution relies solely on visibility.

We illustrate the operation context of the add-wins set from Fig. 2. Event facts in Datalog refer to an event identifier, the operation name, and its parameters. For example, event `1` applies `add` with parameter `a`. For readability, the input relation `vis` contains only

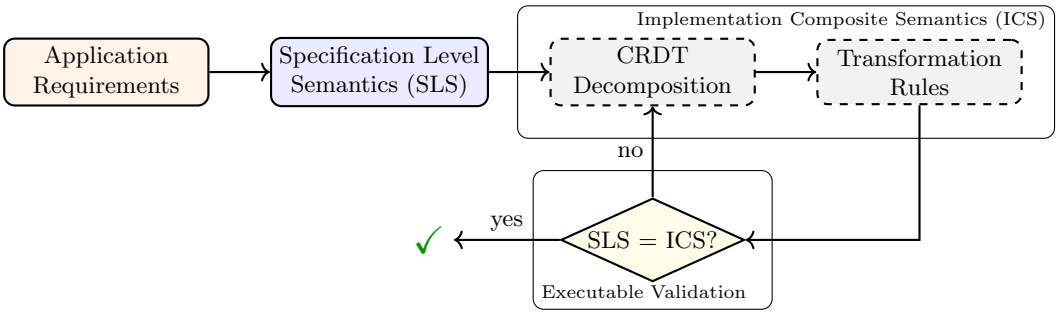


Fig. 3: Workflow for deriving and validating CRDT compositions. Executable validation checks if the implementation compositional semantics (ICS) matches the specification-level semantics (SLS); mismatches require revising the CRDT decomposition.

direct visibility edges, and we compute its transitive closure to obtain full visibility. Events are considered concurrent when they are not connected by the visibility relation.

In general, CRDT correctness is argued via *strong eventual consistency*: replicas that have (eventually) observed the same set of updates must converge to the same state. In our setting, concurrency and update propagation is captured explicitly in the operation context which we assume to be well-formed and identical across replicas once they have observed the same updates. Each data type X is then defined by function $\mathcal{F}_{\mathrm{X}}$ over this context. Consequently, any two replicas that evaluate $\mathcal{F}_{\mathrm{X}}$ on the same operation context derive the same output relations, yielding convergence by construction. Since our Datalog programs implement $\mathcal{F}_{\mathrm{X}}$ as rule evaluation over the relations representing the context, they inherit this determinism directly.

3.2 CRDT Encoding in Datalog

In Section 2, we introduced declarative semantics for basic CRDTs, such as the add-wins set (Equation (1)). In Datalog, we express these semantics as rules that derive one or more output relations representing the observable state at the end of the operation context.

```
postVisOp(E1) :- setEvent(E1, _, X), setEvent(E2, "del", X), visible(E1, E2), E1 \= E2.
setState(X)   :- setEvent(E, "add", X), not postVisOp(E).
```

Listing 1: Add-wins set semantics

Listing 1 gives the Datalog encoding of the add-wins set. Events are represented by `setEvent(E,O,X)`, where `E` is an event identifier, `O` an operation, and `X` a value. The predicate `setState(X)` computes `hasV(X)` over the operation context. A value `X` is in the set if and only if there exists an `add(X)` that is not canceled by a later `del(X)` (`not postVisOp`).

Collaborative applications typically compose basic or nested CRDTs into composite designs (Kuessner et al. 2023). Accordingly, CRDTLog supports nested CRDTs such as maps; our graph case study uses a map-of-sets, described in Section Appendix D.

3.3 Composite CRDTs and CRDT Applications

Collaborative applications typically combine multiple CRDTs into composite ones (Kuessner et al. 2023). We illustrate the workflow on composite CRDTs using the

ID graph application from Table 1. Recall that a node is in the graph if it has been added and not later removed, unless a concurrent `addE` defeats the removal. An edge is present if and only if it has been added and not subsequently removed. We define the corresponding specification-level semantics (SLS) in Equation (2).

$$
\begin{aligned}
&\mathcal{F}_{\text{IDGraph_SLS}}\big(\texttt{hasN}(n), \mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar}\big) = \text{true} \\
&\iff \exists e \in \mathrm{E}:\ \mathrm{op}(e) = \texttt{addN}(n)\ \wedge\ \Big(\neg\exists e' \in \mathrm{E}:\ \mathrm{op}(e') = \texttt{rmvN}(b) \wedge e \xrightarrow{\text{vis}} e' \\
&\qquad \vee\ \exists e', e'' \in \mathrm{E}:\ \mathrm{op}(e') = \texttt{rmvN}(n) \wedge e \xrightarrow{\text{vis}} e' \\
&\qquad\quad \wedge \big(\mathrm{op}(e'') = \texttt{addE}(n, m)\ \vee\ \mathrm{op}(e'') = \texttt{addE}(m, n)\big)\ \wedge e' \not\xrightarrow{\text{vis}} e'' \wedge e'' \not\xrightarrow{\text{vis}} e'\Big) \\
&\mathcal{F}_{\text{IDGraph_SLS}}\big(\texttt{hasE}(n, m), \mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar}\big) = \text{true} \\
&\iff \exists e \in \mathrm{E} : \mathrm{op}(e) = \texttt{addE}(n, m) \wedge \Big(\neg\exists e' \in \mathrm{E} : \mathrm{op}(e') = \texttt{rmvE}(n, m) \wedge e \xrightarrow{\text{vis}} e'\Big)
\end{aligned}
\tag{2}
$$

In the executable SLS in Datalog (Listing 2), we model the input as facts `graphEvent(E, O, V1, V2)`, where `E` is the event identifier, `O` is the operation of the event, and `V1` and `V2` are the parameters of the operation. For a node `n` to be in the resulting graph state `nodes(K)`, there must be an `addN(n)` which is not visible to a `remove` (`not postVisRmvN`); or, if there is a `postVisRmvN`, there exists a concurrent `addE` (`concAddE`), either outgoing or incoming from `n`. An edge is in the resulting graph state (`edges(N,M)`) if the edge has been added and it has not been removed later (`postVisRmvE`).

```
conc(E1,E2) :- graphEvent(E1,_,_,_), graphEvent(E2,_,_,_), not visible(E1,E2), not
     visible(E2,E1), E1\=E2.

concAddE(E2, K) :- graphEvent(E2,_,_,_), graphEvent(E1, "addE", K, _), conc(E1,E2).
concAddE(E2, K) :- graphEvent(E2,_,_,_), graphEvent(E1, "addE", _, K), conc(E1,E2).

postVisRmvN(E1, K, E2) :- graphEvent(E1, _, _, _), graphEvent(E2, "rmvN", K, _),
     visible(E1, E2), E1\=E2, not concAddE(E2, K).

nodes(K) :- graphEvent(E, "addN", K, _), not postVisRmvN(E, K, _).
nodes(K) :- graphEvent(E, "addN", K, _), postVisRmvN(E, K, S), concAddE(S, K).

postVisRmvE(E1) :- graphEvent(E1,_,K1, K2), graphEvent(E2, "rmvE", K1, K2), visible(E1,
     E2), E \= S.

edges(N, M) :- graphEvent(E, "addE", N, M), not postVisRmvE(E).
```

Listing 2: SLS semantics of isolate-delete graph.

The next step is to derive the *implementation-level compositional semantics* (ICS) by choosing a CRDT decomposition and defining transformation rules (Definition 2). A natural decomposition mirrors a sequential design: a replicated node set (`NodeSet`) and a map from each node to its outgoing-edge set (`EdgeMap`). The transformation rules interpret each application event as one or more component updates (Fig. 4). For nodes, `addN(n)` and `rmvN(n)` translate to `add(n)` and `del(n)`. To realize isolate-delete under concurrency, a concurrent `addE` must be able to defeat a node removal; we therefore translate `addE(n,m)` into additional `add(n)` and `add(m)` updates on the `NodeSet`, inducing the required conflict at the node component. For edges, we store outgoing edges in an adjacency map: `addE(n,m)` translates to `upd(n,add(m))`, and `rmvE(n,m)` to `upd(n,del(m))`. We use `upd` (rather than removing key `n`) because key removal would delete all outgoing edges of `n`, not just `(n,m)`. Since `EdgeMap`is a map-of-sets, its internal transformation rules are applied as well; the full definition of map-of-sets is provided in the Section Appendix D.

$$
\begin{aligned}
\texttt{IDGraph.addN}(n) &\overset{\mathcal{T}}{\Rightarrow} \texttt{NodeSet.add}(n) \\
\texttt{IDGraph.rmvN}(n) &\overset{\mathcal{T}}{\Rightarrow} \texttt{NodeSet.del}(n) \\
\texttt{IDGraph.addE}(n,m) &\overset{\mathcal{T}}{\Rightarrow} \{\texttt{NodeSet.add}(n), \texttt{NodeSet.add}(m), \texttt{EdgeMap.upd}(n, \texttt{add}(m)) \\
\texttt{IDGraph.rmvE}(n,m) &\overset{\mathcal{T}}{\Rightarrow} \texttt{EdgeMap.upd}(n, \texttt{del}(m)) \\
\texttt{EdgeMap.upd}(n, \texttt{add}(m)) &\overset{\mathcal{T}}{\Rightarrow} \{\ \texttt{KeySet.add}(n),\ \texttt{ValSet}_n\texttt{.add}(m)\ \} \\
\texttt{EdgeMap.upd}(n, \texttt{del}(m)) &\overset{\mathcal{T}}{\Rightarrow} \{\ \texttt{KeySet.add}(n), \texttt{ValSet}_n\texttt{.del}(m)\}
\end{aligned}
$$

Fig. 4: Layered transformation for isolate-delete graph.

Definition 2 (*Transformation rules*)
A *transformation rule* is a function $\mathcal{T} : Op_X \to \mathcal{P}(Op_{X_1} \cup \cdots \cup Op_{X_2})$ that maps operations of a data type X to operations of the components $X_1, \ldots, X_n$ to implement X.

The resulting ICS of the ID graph is given as:

$$
\begin{aligned}
&\mathcal{F}_{\text{IDGraph_ICS}}\big(\texttt{hasN}(n), (\mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar})\big) = \mathrm{true} \\
&\quad \iff \mathcal{F}_{\text{Set}}\big(\texttt{hasV}(n), (\mathrm{E}_{\texttt{NodeSet}}, \mathrm{op}_{\texttt{NodeSet}}, \mathrm{vis}, \mathrm{ar})\big) \\
&\mathcal{F}_{\text{IDGraph_ICS}}\big(\texttt{hasE}(n,m), (\mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar})\big) = \mathrm{true} \\
&\quad \iff \mathcal{F}_{\text{MapSet}}\big(\texttt{hasKV}(n,m), (\mathrm{E}_{\texttt{EdgeMap}}, \mathrm{op}_{\texttt{EdgeMap}}, \mathrm{vis}, \mathrm{ar})\big)
\end{aligned}
\tag{3}
$$

In the Datalog realization (Listing 3), the input facts have the same signature as the SLS, namely `graphEvent(E, O, V1, V2)`. The resulting state is computed for each component. Note that this implementation may produce keys with empty value sets. We interpret such entries as non-existent edges and ignore them when inspecting the application state.

```
nodesEvent(E, "add", V) :- graphEvent(E, "addN", V, _).
nodesEvent(E, "rmv", V) :- graphEvent(E, "rmvN", V, _).
nodesEvent(E, "add", V) :- graphEvent(E, "addE", V, _).
nodesEvent(E, "add", V) :- graphEvent(E, "addE", _, V).

edgesMapSetEvent(E, "put", "add", V1, V2) :- graphEvent(E, "addE", V1, V2).
edgesMapSetEvent(E, "put", "rmv", V1, V2) :- graphEvent(E, "rmvE", V1, V2).
```

Listing 3: ICS for isolate-delete graph

CRDT composition is not semantics-neutral: decomposition choices determine which operations conflict and how conflicts are resolved. For graphs, nodes and edges are separate structures but semantically coupled; i.e., an `addE` may defeat a concurrent `rmvN`, so the transformation rules must intentionally induce or avoid conflicts.

4 A Case Study on Graph Applications

To evaluate CRDTLog, we consider collaborative graph applications as a representative case study. We instantiate the framework on two directed graph variants: the *isolate-delete (ID)* graph introduced in Section 2 and discussed in Section 3, and a *detach-delete (DD)* graph that we introduce below. These applications capture common deletion policies in collaborative graph editing and exercise non-trivial interactions between collaborating users. Our evaluation pursues two main goals. First, we demonstrate that the framework supports precise and executable semantics for graph-based applications. Second, we show how these semantics guide the systematic selection of CRDT components and the design of the transformation rules.

Isolate-delete semantics (ID). We first revisit the ID semantics defined in Section 3.3. A natural implementation-level decomposition (ICS) represents nodes as an add-wins set and outgoing edges as a map-of-sets. The transformation layer is straightforward for most operations and maps `addE` to additional `add` updates on the node set to ensure that concurrent edge additions override node removals, as required by the SLS.

Detach-delete semantics (DD). We next consider the DD semantics for directed graphs. This specifies that removing a node also removes all its incident edges, ensuring the absence of dangling edges. The formal definition differs from ID only in the definition of $\texttt{hasE}(u, v)$ (Table 1). DD preserves the node semantics of ID. In particular, a node n is present if it has been added and not subsequently removed, unless the removal is invalidated by a concurrent edge addition involving n. Hence, the definition of $\mathcal{F}_{\text{DDGraph_SLS}}(\texttt{hasN}(n), \dots)$ is identical to the isolate-delete case (Eq. 2). Further, a DD graph contains an edge `(n,m)` if it has been added, has not been subsequently removed, and neither the source nor target node has been removed. The node condition is the new part: if `n` (or `m`) is removed, then all incident edges are removed as well (Eq. 4).

$$\begin{aligned}
&\mathcal{F}_{\text{DDGraph_SLS}}\big(\texttt{hasE}(n,m), \mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar}\big) = \text{true} \\
&\quad \iff \exists e \in \mathrm{E}: \ \mathrm{op}(e) = \texttt{addE}(n,m) \ \wedge \\
&\qquad \neg\exists e' \in E: \ \mathrm{op}(e') = \texttt{rmvE}(n,m,\wedge) e \xrightarrow{\text{vis}} e' \ \wedge \big(\Gamma(e,n) \wedge \Gamma(e,m)\big) \\
&\Gamma(e,u) \stackrel{\text{def}}{=} \neg\exists s \in \mathrm{E}: \ \mathrm{op}(s) = \texttt{rmvN}(u) \wedge e \xrightarrow{\text{vis}} s
\end{aligned} \tag{4}$$

The corresponding Datalog rules can be found in Listing 4.

At first glance, one might attempt to reuse the ID implementation strategy, using a set for nodes and a map-of-sets for edges. Thus, removing a node `n` would remove `n` from the node set and delete its outgoing edges by removing the key `n` from the edge map. Incoming edges could then be removed by deleting the corresponding entries from the value sets associated with other keys. However, this straightforward approach fails because it introduces unintended conflicts in the map. In particular, a concurrent `rmvE`(n,m) and `rmvN`(n) generate conflicting updates that combine a value-level update (a `upd` or inner-set modification) with a key-level `rmv`. Under update-wins map semantics, the `upd` may resurrect the key `n`, thereby preserving outgoing edges from `n`. The resulting state contains dangling edges, which violates the DD semantics. To avoid this, we represent the graph with one set for nodes and one set for edges. The edge set stores tuples `(n,m)`. This design eliminates key-level put-versus-remove conflicts since all edge updates are applied uniformly to a single set. The transformations for node operations are unchanged from the ID case. The transformation for edge operations is defined in Fig. 5. The compositional

$$\begin{aligned}
\texttt{DDGraph.rmvN}(n) &\stackrel{\mathcal{T}}{\Rightarrow} \{\texttt{EdgeSet.del}(n,m), \texttt{EdgeSet.del}(m,n)\}, \\
&\qquad \text{where } m \text{ is a node to/from which an edge from/to } n \text{ exists.} \\
\texttt{DDGraph.addE}(n,m) &\stackrel{\mathcal{T}}{\Rightarrow} \texttt{EdgeSet.add}(n,m) \\
\texttt{DDGraph.rmvE}(n,m) &\stackrel{\mathcal{T}}{\Rightarrow} \texttt{EdgeSet.del}(n,m)
\end{aligned}$$

Fig. 5: Transformation rules for detach-delete graph.

semantics is given in Eq. 5, and the Datalog semantics can be found in Listing 5.

$$\begin{aligned}&\mathcal{F}_{\text{DDGraph_ICS}}\big(\texttt{hasE}(n,m),(\text{E},\text{op},\text{vis},\text{ar})\big)=\text{true}\\&\quad\iff \mathcal{F}_{\text{Set}}\big(\texttt{hasV}(n,m),(\text{E}_{\texttt{EdgeSet}},\text{op}_{\texttt{EdgeSet}},\text{vis},\text{ar})\big)\end{aligned}\tag{5}$$

This case study shows the value of explicit semantic specifications: they expose when a CRDT decomposition enforces conflict resolution incompatible with application intent. CRDTLOG makes this mismatch explicit at the specification level and guides designers toward a composition with the desired semantics.

5 Experimental Evaluation

For the experimental evaluation, we compare the implementation-level compositional semantics (ICS) with the specification-level semantics (SLS) for the two graph applications from Section 4, yielding four implementations in total (SLS and ICS for ID and DD). Correctness is assessed via property-based testing: for eight configurations, we generate 1K independent executions and verify that SLS and ICS produce identical output graphs. We then study scalability by benchmarking semantic evaluation time while varying the number of events, replicas, and graph size.

The evaluation quantitatively assesses whether the CRDT-based implementations faithfully realize the abstract specifications and how the evaluation cost evolves as workload parameters increase. We structure this analysis around the research questions below.

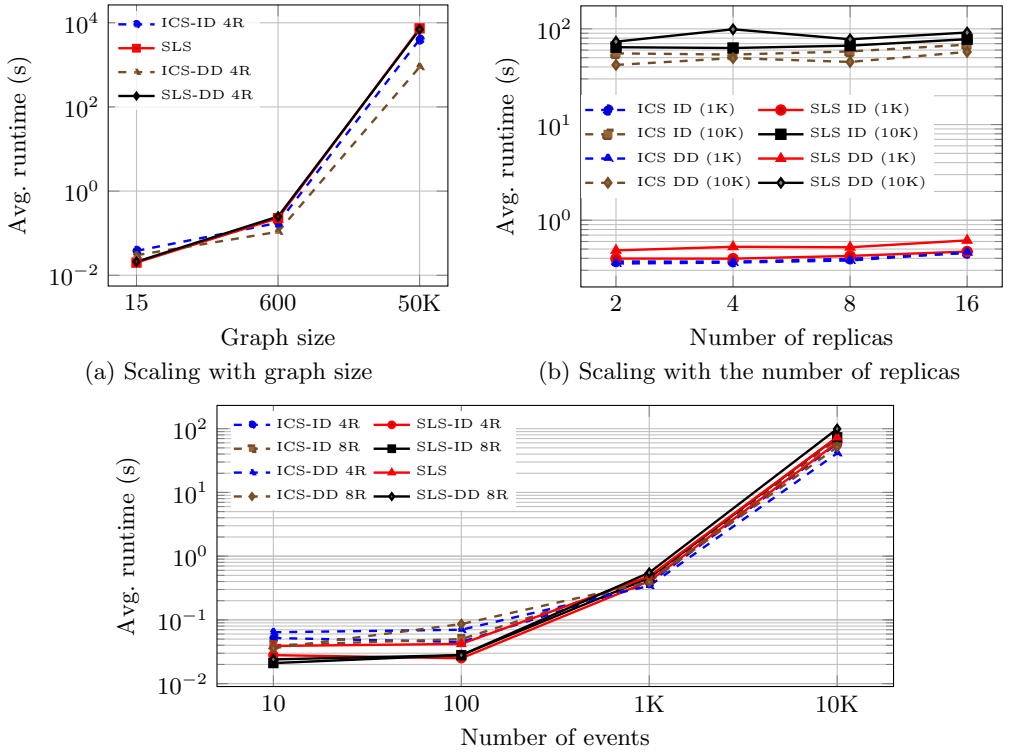
- ***RQ1*** *(Semantic Equivalence).* Do ICS and SLS yield the same observable states for all tested operation contexts?
- ***RQ2*** *(Application Graph Scalability).* How does the size of the application graph affect the cost of semantic evaluation?
- ***RQ3*** *(Concurrency Scalability).* What is the impact of increasing the replica count and concurrency on evaluation cost?
- ***RQ4*** *(Event Scalability).* How does evaluation cost scale with increasing events?

Experimental Setup. All experimental inputs were automatically generated using Python scripts that enforce each graph application's preconditions. For example, in the isolate-delete variant, `removeNode(n)` is never generated for nodes with incident edges, ensuring that all executions satisfy the assumptions of the model.

Equivalence tests ran on a machine running Ubuntu 22.04.2 with 11th Gen Intel(R) Core(TM) i7-1165G7 2.80 GHz CPU and 32 GB of memory.

Scalability experiments ran on a CPU-only x86_64 server equipped with two AMD EPYC 7443 processors. For the experiments, 100 GB of main memory was allocated. The CPU frequency boost was enabled (reported range: 1.5–2.85 GHz). For each scalability configuration, we executed 30 independent runs and reported the mean execution times. Both SLS and ICS implementations were evaluated on the same (generated) inputs.

*Semantic Equivalence (**RQ1**).* SLS and ICS compute identical observable states for all tested operation contexts. We evaluated semantic equivalence using a property-based testing approach (Claessen and Hughes 2000), generating randomized operation contexts and comparing the outputs of SLS and ICS. We considered eight configurations, combin-



(a) Scaling with graph size

(b) Scaling with the number of replicas

(c) Scaling with the number of events

Fig. 6: Scalability of SLS and ICS for isolate-delete (ID) and detach-delete (DD): (a) graph size, (b) number of replicas, and (c) number of events

ing two replica counts (5 and 10) with four event volumes (20, 50, 100, and 1K events). For each configuration, we generated 1,000 independent test inputs. All executions terminated successfully, and in every case, the SLS and ICS implementations produced identical results. These results provide empirical evidence that our CRDT compositions faithfully realize the intended semantics for both the ID and DD graph variants. Average execution times per test are reported in the extended version.

*Scalability (**RQ2**–**RQ4**).* We analyze how semantic evaluation cost scales with operation context size, concurrency, and application graph size. Figure 6 reports runtime as a function of graph size (Fig. 6a), replica count (Fig. 6b), and number of events (Fig. 6c).

***RQ2** (Application Graph Scalability).* Evaluation cost increases with application graph size (nodes and edges), but ICS scales better than SLS for larger graphs. To study graph size scalability, we sampled random graphs with a fixed number of nodes (10, 100, and 1K) using an edge probability of 0.05, yielding an expected number of edges proportional to graph size. For each graph, we generated an operation context with 4 replicas using only `addNode` and `addEdge` events. Concurrency is modeled by allowing operations to interleave and branch, producing non-trivial visibility structures.

Figure 6a shows that for small graphs, ICS implementations are slightly slower than the corresponding SLS specifications due to the overhead of transformation rules. However,

as graph size increases (and with it the number of events in the operation context), ICS becomes consistently faster for both applications. At approximately 50K graph entities, the ICS realization is nearly twice as fast as SLS for ID and close to an order of magnitude faster for DD. This behavior reflects the lower rule complexity of the component CRDT semantics used in ICS, which outweighs the transformation overhead once evaluation is dominated by joins over many events.

RQ3 *(Concurrency Scalability).* Increasing the number of replicas only mildly impacts evaluation cost. As shown in Figure 6b, increasing the replica count results in only modest runtime growth. In CRDTLog, the number of replicas mainly affects the branching structure of the visibility relation without substantially increasing the complexity of rule evaluation. This suggests that semantic evaluation scales well with respect to concurrency.

RQ4 *(Event Scalability).* Evaluation cost grows primarily with the number of events in the operation context. As summarized in Figure 6c, runtime increases steadily as the number of events grows for both graph variants and both semantic encodings. This trend holds largely independently of the number of replicas, indicating that event volume is the dominant factor driving semantic evaluation cost.

Discussion. While we do not directly compare ID and DD semantics, their scaling behavior differs markedly. In the large-graph regime, DD remains substantially faster, and the gap between SLS and ICS is most pronounced there. A plausible explanation lies in the underlying representations: the ID realization stores edges in a map-of-sets, introducing nesting, additional derived relations, and multiple component-level updates per graph event. In contrast, the DD realization uses a single set of edge tuples, so each graph event maps to at most one component update. This highlights that decomposing an application into simpler CRDT components can significantly reduce intermediate joins and improve evaluation time, even when the resulting semantics are identical.

6 Conclusions and Perspectives

We introduced CRDTLog, a declarative framework for specifying and reasoning about the semantics of replicated data types and collaborative applications over operation contexts. The framework cleanly separates specification-level semantics (SLS) from implementation-level compositional semantics (ICS), obtained via CRDT composition and explicit transformation rules. Both semantics are implemented in Datalog, with modular support for basic, nested, and composite CRDTs.

We evaluated CRDTLog on two collaborative graph applications (isolate-delete and detach-delete). Property-based testing showed that ICS consistently matches SLS across all tested configurations, providing empirical evidence of semantic equivalence. Scalability results indicate that the evaluation cost is driven mainly by execution size, with replica count having only a limited impact. For larger workloads, ICS often outperforms SLS, making it particularly well suited for large-scale testing and semantic prototyping.

As future work, we plan to formalize CRDTLog in a theorem prover to enable machine-checked proofs of additional semantic properties.

References

Peter Alvaro, William R. Marczak, Neil Conway, and et al. Dedalus: Datalog in time and space. In *Datalog*, volume 6702 of *LNCS*, pages 262–281. Springer, 2010. doi: 10.1007/978-3-642-24206-9_16. URL `https://doi.org/10.1007/978-3-642-24206-9_16`.

Peter Alvaro, Neil Conway, Joseph M. Hellerstein, and William R. Marczak. Consistency analysis in bloom: a CALM and collected approach. In *CIDR*, pages 249–260. www.cidrdb.org, 2011. URL `http://cidrdb.org/cidr2011/Papers/CIDR11_Paper35.pdf`.

Evelyn Borth, Philipp Lersch, and Annette Bieniusa. Directed acyclic graph crdts. In *PaPoC@EuroSys*, pages 30–37. ACM, 2025.

Sebastian Burckhardt. Principles of eventual consistency. *Found. Trends Program. Lang.*, 1(1-2): 1–150, 2014. doi: 10.1561/2500000011. URL `https://doi.org/10.1561/2500000011`.

Koen Claessen and John Hughes. Quickcheck: a lightweight tool for random testing of haskell programs. In *ICFP*, pages 268–279. ACM, 2000.

Seth Gilbert and Nancy Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *Acm Sigact News*, 33(2):51–59, 2002.

Herbert Jordan, Bernhard Scholz, and Pavle Subotic. Soufflé: On synthesis of program analyzers. In *CAV (2)*, volume 9780 of *Lecture Notes in Computer Science*, pages 422–430. Springer, 2016a.

Herbert Jordan, Bernhard Scholz, and Pavle Subotić. Soufflé: On synthesis of program analyzers. In Swarat Chaudhuri and Azadeh Farzan, editors, *Computer Aided Verification*, pages 422–430, Cham, 2016b. Springer International Publishing. ISBN 978-3-319-41540-6.

Martin Klepmann. Automerge. `https://github.com/automerge/automerge` (visited: 2026-01), 2017.

Christian Kuessner, Ragnar Mogk, Anna-Katharina Wickert, and Mira Mezini. Algebraic replicated data types: Programming secure local-first software. In *37th European Conference on Object-Oriented Programming (ECOOP 2023)*, pages 14–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023.

Lindsey Kuper and Ryan R. Newton. Lvars: lattice-based data structures for deterministic parallelism. In *FHPC@ICFP*, pages 71–84. ACM, 2013. doi: 10.1145/2502323.2502326. URL `https://doi.org/10.1145/2502323.2502326`.

Shadaj Laddad, Conor Power, Mae Milano, Alvin Cheung, Natacha Crooks, and Joseph M. Hellerstein. Keep CALM and CRDT on. *Proc. VLDB Endow.*, 16(4):856–863, 2022. doi: 10.48550/arXiv.2210.12605. URL `https://doi.org/10.48550/arXiv.2210.12605`.

Christopher Meiklejohn and Peter Van Roy. Lasp: a language for distributed, coordination-free programming. In *PPDP*, pages 184–195. ACM, 2015. doi: 10.1145/2790449.2790525. URL `https://doi.org/10.1145/2790449.2790525`.

Petru Nicolaescu, Kevin Jahns, Michael Derntl, and Ralf Klamma. Yjs: A framework for near real-time P2P shared editing on arbitrary data types. In *15th International Conference on Web Engineering*, ICWE 2015, pages 675–678. Springer LNCS volume 9114, June 2015. doi: 10.1007/978-3-319-19890-3_55. URL `http://dbis.rwth-aachen.de/~derntl/papers/preprints/icwe2015-preprint.pdf`.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems*, pages 386–400. Springer, 2011.

Vimala Soundarapandian, Adharsh Kamath, Kartik Nagar, and KC Sivaramakrishnan. Certified mergeable replicated data types. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 332–347, 2022.

Appendix A Implementation details

This appendix complements the main body by providing the full Datalog encodings underlying the detach-delete (DD) graph semantics introduced in Section 4.

We provide the remaining SLS rules for the DD graph, characterizing when edges survive concurrent node deletions and preventing dangling edges.

```
postVisRemoveNode(E, K, S) :- graphEvent(E, _, _, _), graphEvent(S, "rmvN", K, _),
     visible(E, S), E \= S.

postVisRemoveEdge(E) :- graphEvent(E, _, K1, K2), graphEvent(S, "rmvE", K1, K2),
     visible(E, S), E \= S.

concurrentAddEdge(E, K) :- graphEvent(E, _, _, _), graphEvent(S, "addE", K, _), not
     visible(E, S), not visible(S, E), E \= S.
concurrentAddEdge(E, K) :- graphEvent(E, _, _, _), graphEvent(S, "addE", _, K), not
     visible(E, S), not visible(S, E), E \= S.

edges(N, M) :- graphEvent(E, "addE", N, M), not postVisRemoveEdge(E), not
     postVisRemoveNode(E, N, _), not postVisRemoveNode(E, M, _).
```

Listing 4: SLS for DD graph (edge rules).

In Listing 4, `edges(N,M)` holds for added edges whose endpoints were not removed, while `postVisRemoveNode` captures concurrent `addEdge` that invalidate node removals.

We then present the corresponding implementation-level compositional semantics (ICS) rules. These rules encode the transformation-based realization of the DD graph using CRDT components, making explicit how node and edge operations are translated and how incident edges are removed upon successful node deletion. The additional helper predicates isolate precisely those edges that must be removed to preserve the detach-delete invariant while respecting the concurrency semantics inherited from the node set.

```
edgesEvent(E, "add", V1, V2) :- graphEvent(E, "addE", V1, V2).
edgesEvent(E, "del", V1, V2) :- graphEvent(E, "rmvE", V1, V2).
edgesEvent(E, "del", V1, V2) :- graphEvent(E, "rmvN", V1, _), preVisAddOutE(E, V2).
edgesEvent(E, "del", V2, V1) :- graphEvent(E, "rmvN", V1, _), preVisAddInE(E, V2).

preVisAddInE(E, V2):-graphEvent(E, _, V1, _),graphEvent(S, "addE", V2, V1),visible(S,
     E),E\=S.
preVisAddOutE(E, V2):-graphEvent(E, _, V1, _),graphEvent(S, "addE", V1, V2),visible(S,
     E),E\=S.
```

Listing 5: ICS for DD graph. Translation rules for the edges.

Note that the two additional predicates `preVisAddIncomingEdge` and `preVisAddOutgoingEdge` identify the edges that are visible during the removal event. The translation then removes exactly these edges, ensuring that successful node deletions detach all incident edges while preserving the intended concurrency behavior captured by the node semantics

Listings 4 and 5 complete the executable definition of the DD graph used in the evaluation. They also illustrate how the abstract workflow described in Figure 3 is instantiated in practice: the SLS rules capture the intended application semantics directly, while the ICS rules operationalize these semantics through CRDT composition and transformation.

Appendix B Evaluation details

This section reports additional details of the experimental evaluation summarized in Section 5. In particular, it provides the full quantitative results of the semantic equivalence tests between specification-level semantics (SLS) and implementation-level compositional semantics (ICS) for both graph variants. The table reports average execution times across configurations, complementing the scalability plots in the main text and supporting the empirical claims about correctness and performance.

Table B 1: Semantic Equivalence. Avg. execution time (s) per run, 1K runs/ category.

#Ops	ICS ID 5R	SLS ID 5R	ICS DD 5R	SLS DD 5R	ICS ID 10R	SLS ID 10R	ICS DD 10R	SLS DD 10R
20	0.029	0.015	0.018	0.015	0.023	0.012	0.017	0.014
50	0.018	0.011	0.017	0.015	0.025	0.014	0.018	0.016
100	0.031	0.020	0.019	0.017	0.025	0.016	0.019	0.018
1000	0.377	0.485	0.343	0.531	0.454	0.501	0.383	0.555

Appendix C Proof details

This section presents formal proofs that both the SLS and ICS of the detach-delete graph preserve the no-dangling-edges invariant under all valid operation contexts.

Lemma 1
For the SLS of the detach-delete (DD) graph, the invariant of no dangling edges holds for any valid operation context.

Proof
Let $C = (E, op, vis, ar)$ be a well-formed operation context satisfying the input preconditions of the DD graph (V, A), i.e., $\texttt{addE}(u,v)$ and $\texttt{rmvN}(n)$ are issued only if n, u, and v are observed at the issuing replica. We show the resulting graph has no dangling edges.

A dangling edge can only arise due to a node-removal event, so we analyze executions in which some $\texttt{rmvN}(n)$ occurs. We fix such a node n and proceed by case distinction. Throughout, we rely on Equation (4): an edge (u, v) is present only if it is added, not visibly removed, and $\Gamma(e, u) \wedge \Gamma(e, v)$ holds for the corresponding add-edge event e.

Case 1. The user removes node n, and there is no concurrent $\texttt{addE}(n,_)$ or $\texttt{addE}(_,n)$.

For this case, we need to consider three subcases: if n has no incident edges, if n has outgoing edges, and if n has incoming edges.

- If n has no outgoing edges, then for every (u, v), we have $u \neq n$ and $v \neq n$, hence no edge can be incident to n in the resulting state. Thus, the invariant holds.
- n has outgoing edges. Since $\texttt{rmvN}(n)$ exists and is visible after the add-edge event along that branch, $\Gamma(e, n)$: $\neg\exists s \in E : \ \mathrm{op}(s) = \mathsf{removeNode}(n) \wedge e \xrightarrow{\text{vis}} s$, is false. This contradicts $(n, m) \in A$. Hence, no edge outgoing from n is present in

the resulting state, so no dangling edge arises. This holds for any outgoing edge candidate, (n, m)
- n has incoming edges. This subcase is analogous to the previous one.

Case 2. User B removes node n, while concurrently user C adds an edge incident to n (i.e., $\texttt{addE}(n, m)$ or $\texttt{addE}(m, n)$).

Let s be a remove-node event with $\text{op}(s) = \texttt{rmvN}(n)$, and let s' be a concurrent add-edge event incident to n as above. Then $\Gamma(e, n)$ evaluates to false for any edge to and from n that is not (n, m). For (n, m) it would evaluate to true, as the $\texttt{rmvN}(n)$ is concurrent and not visible. In this situation, however, the node predicate **??** also keeps n present: if a visible $\texttt{rmvN}(n)$ exists but there is a concurrent incident add-edge, then $\mathcal{F}_{\text{GraphDD_SLS}}(\texttt{hasN}(n) = \texttt{true}$. Hence, $n \in V$ and the edge (n, m) will be in the resulting state, but none of the previous edges to and from n.Therefore, the edge (n, m) that remains incident to n is not dangling.

Finally, concurrent removals of the same node, removals of distinct nodes, or removals interleaved with additions of unrelated nodes do not introduce additional ways of creating an edge whose endpoint is absent: edge presence is always guarded by the endpoint conditions $\Gamma(e, \cdot)$ in Equation (4) together with the node predicate **??**. Therefore, no resulting state from a operation context contains a dangling edge. □

Lemma 2
For the ICS of the detach-delete (DD) graph, the invariant of no dangling edges holds for any valid operation context.

Proof
Let $C = (E, op, vis, ar)$ be a well-formed operation context satisfying the input preconditions of the DD graph (in particular, $\texttt{addE}(u,v)$ and $\texttt{rmvN}(n)$ are issued only if n, u, and v are observed at the issuing replica). We show that the resulting graph has no dangling edges.

A dangling edge can only arise due to a node-removal event, so we analyze executions in which some $\texttt{rmvN}$(n) occurs. We fix such a node n and proceed by case distinction. Throughout, we rely on **??** and Equation (5): an edge (u, v) is present only if it is in the edges set.

Case 1. The user removes node n and there is no concurrent $\texttt{addE}(n, _)$ or $\texttt{addE}(_, n)$. We again consider three subcases: if n has no incident edges, if n has outgoing edges, and if n has incoming edges.

- If n has no incident edges, then for every (u, v), we have $u \neq n$ and $v \neq n$; hence, no edge can be incident to n in the resulting state. Thus, the invariant holds.
- n has outgoing edges.
  Each $\texttt{rmvN}(n)$ is translated to a $\texttt{del}(n, m)$ for each outgoing edge in the underlying set, thus removing all outgoing edges. As there is no concurrent operation that adds anything to the set, there is no `add` to win over the removal. Thus, the resulting state has no dangling edges.

- n has incoming edges.
  This case is analogous to the previous one; $\texttt{rmvN}(n)$ is also translated to a $\texttt{del}(m, n)$ for every incoming edge.

Case 2. The user removes node n, while concurrently, user B adds an edge incident to n (i.e., $\texttt{addE}(n, m)$ or $\texttt{addE}(m, n)$.
$\texttt{rmvN}(n)$ would translate to a $\texttt{del}(n)$ in the nodes set. $\texttt{addE}(n,m)$ will translate to $\{$ $\texttt{add}(n)$, $\texttt{add}(m)\}$ within the nodes set. In the concurrency conflict between $\texttt{del}(n)$ and $\texttt{add}(n)$, the latter would win. Thus, the resulting state would contain n; therefore, this case also does not cause dangling edges.
Case 3. User B removes node n, while concurrently, user C removes an edge between nodes n and m.
In this case, the removal of n would be successful, as $\texttt{rmvE}(n,m)$ does not have a transformation into the nodes set. This case would not cause any conflicts (add vs. remove) in the edges set CRDT either, because concurrent removals do not conflict. Thus, the resulting state will contain neither n nor any dangling edges from or to n.

Finally, concurrent removals of the same node, removals of distinct nodes, or removals interleaved with additions of unrelated nodes do not introduce additional ways of creating an edge whose endpoint is absent. Hence, no resulting state from an operation context can contain a dangling edge. □

Appendix D Nested CRDTs

Our goal is to model applications that rely on both basic and derived CRDTs. Accordingly, the framework is modular: designers obtain nested data types by choosing component CRDTs and writing transformation rules from top-level events to component events. In nested structures, the decomposition is often structural. For example, a map-of-sets combines a set that tracks keys and a set of values for each key.

To make this example concrete, consider a map-of-sets with $Op_X = \{\texttt{upd}(k, v), \texttt{rmv}(k)\}$, where v can be either $\texttt{add}(x)$ or $\texttt{del}(x)$. The first step is to define the *specification level semantics* (SLS) that formalizes the behavior of the application as a single object:

$$\begin{aligned}
&\mathcal{F}_{\text{MapSet_SLS}}\big(\texttt{hasKV}(k, x), (\mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar})\big) = \text{true} \\
&\iff \exists e \in \mathrm{E} : \mathrm{op}(e) = \texttt{upd}(k, \texttt{add}(x)) \wedge \big(\neg\exists e' \in \mathrm{E} : \mathrm{op}(e') = \texttt{rmv}(k) \wedge e \xrightarrow{vis} e'\big) \\
&\quad \wedge \big(\neg\exists e'' \in \mathrm{E} : \mathrm{op}(e'') = \texttt{upd}(k, \texttt{del}(x)) \wedge e \xrightarrow{vis} e''\big)
\end{aligned} \tag{D1}$$

A map-of-sets contains a key `k` and value `x` in the set associated with `k` if it was inserted by a `put` operation and neither the key nor the value were later removed.

Next, we obtain the ICS for the map-of-set. Intuitively, the map contains (k, x) when k is present in the key set and x is present in the value set associated with k:

$$\begin{aligned}
&\mathcal{F}_{\text{MapSet_ICS}}\big(\texttt{hasKV}(k, x), (\mathrm{E}, \mathrm{op}, \mathrm{vis}, \mathrm{ar})\big) = \text{true} \iff \\
&\quad \mathcal{F}_{\text{Set}}\big(\texttt{hasV}(k), (\mathrm{E}, \mathrm{op}_{\texttt{KeySet}}, \mathrm{vis}, \mathrm{ar})\big) \wedge \mathcal{F}_{\text{set}_k}\big(\texttt{hasV}(x), (\mathrm{E}_{\texttt{ValSet}}, \mathrm{op}_{\texttt{ValSet}}, \mathrm{vis}, \mathrm{ar})\big)
\end{aligned} \tag{D2}$$

Figure D 1 presents the corresponding transformation rules.

In Datalog, the event facts of the map-of-set `mapSetOp(E,Op,InnerOp,K,X)` consist of the event identifier `E`, the map-level operation `Op`, the inner set operation `InnerOp` and the operation parameters `K` and `X` (Listing 6). `keysOp` and `setOp` correspond to the transformation

$$\begin{aligned}
\texttt{MapSet.upd}(k, \texttt{add}(v)) &\overset{\mathcal{T}}{\Rightarrow} \{\texttt{KeySet.add}(k), \texttt{ValSet.add}(v)\} \\
\texttt{MapSet.upd}(k, \texttt{del}(v)) &\overset{\mathcal{T}}{\Rightarrow} \{\texttt{KeySet.add}(k), \texttt{ValSet.del}(v)\} \\
\texttt{MapSet.rmv}(k) &\overset{\mathcal{T}}{\Rightarrow} \texttt{KeySet.del}(k)
\end{aligned}$$

Fig. D 1: Layered transformation for update-wins map-of-sets

rules. `mapState` computes the final state. Note that a key might exist with an empty value set. For that we use the empty value `""`.

```
keysEvent(E, "add", K) :- mapSetEvent(E, "upd",  _, K, _).
keysEvent(E, "del", K) :- mapSetEvent(E, "rmv", _, K, _).

setEvent(E, InnerOp, X, K) :- mapSetOp(E, "upd", InnerOp, K, X), not postVisRemove(E).

mapState(K, X)  :- keysState(K), setState(K, X).
mapState(K, "") :- keysState(K), not setState(K, _).
```

Listing 6: Map-of-set semantics expressed in Datalog.

Soufflé does not support dynamic instantiation of modules at runtime. In particular, a map-of-sets cannot allocate a fresh set instance per key during evaluation. We therefore encode all inner sets within a single indexed set instance, using an additional identifier (the key) to distinguish per-key state. The corresponding code can be found in listing 7.

```
laterVisOpSet(E1, P) :- setEvent(E1, _, X, S), setEvent(E2, "del", X, S), visible(E1,
    E2), E1\=E2.

lastVisibleOpSet(E1) :- setEvent(E1, _, _, _), not laterVisOpSet(E1, E2).

setState(S, X) :- setEvent(E, "add", X, S), lastVisibleOpSet(E).
```

Listing 7: Value set semantics expressed for ICS map-of-set

With this encoding pattern, deeper nesting (e.g. maps of maps) reduces to the repeated use of transformation rules and identifiers.